

Concurrency and event collection

The final form of flow control you can implement in workflows is concurrent execution.

 This allows you to efficiently run long-running tasks in parallel, and gather them together when they are needed. This can also let you perform map-reduce style tasks.

To do this, we'll be using the `Context` object that we already encountered when working with `AgentWorkflow`. The `Context` is available to every step in a workflow: to access it, declare it as an argument to your step and it will be automatically populated.

In this example, we use a new method, `Context.send_event` rather than returning an event. This allows us to emit multiple events in parallel rather than returning just one as we have previously.

! Note: Run the hidden cells below before running the rest of the code. !

```
!pip install llama-index -q -q
```

```

import asyncio
import random
from llama_index.core.workflow import (
    Context,
    Event,
    StartEvent,
    StopEvent,
    Workflow,
    step,
)

class StepTwoEvent(Event):
    query: str

class ParallelFlow(Workflow):
    @step
    async def start(self, ctx: Context, ev: StartEvent) -> StepTwoEvent:
        ctx.send_event(StepTwoEvent(query="Query 1"))
        ctx.send_event(StepTwoEvent(query="Query 2"))
        ctx.send_event(StepTwoEvent(query="Query 3"))

    @step(num_workers=4)
    async def step_two(self, ctx: Context, ev: StepTwoEvent) -> StopEvent:
        print("Running slow query ", ev.query)
        await asyncio.sleep(random.randint(1, 5))

        return StopEvent(result=ev.query)

```

```

w = ParallelFlow(timeout=10, verbose=False)
result = await w.run(message="Start the workflow.")
print(result)

```

```

Running slow query  Query 1
Running slow query  Query 2
Running slow query  Query 3
Query 3

```

As you can see, it executes all 3 queries. The first one to complete emits a `StopEvent`, at which point the workflow halts without waiting for the other 2 events.



Collecting events

But what if we do want the output of all 3 events? Another method, `Context.collect_events`, exists for that purpose:

```

class StepThreeEvent(Event):
    result: str

class ConcurrentFlow(Workflow):
    @step
    async def start(self, ctx: Context, ev: StartEvent) -> StepTwoEvent:
        ctx.send_event(StepTwoEvent(query="Query 1"))
        ctx.send_event(StepTwoEvent(query="Query 2"))
        ctx.send_event(StepTwoEvent(query="Query 3"))

    @step(num_workers=4)
    async def step_two(self, ctx: Context, ev: StepTwoEvent) -> StepThreeEvent:
        print("Running query ", ev.query)
        await asyncio.sleep(random.randint(1, 5))
        return StepThreeEvent(result=ev.query)

    @step
    async def step_three(self, ctx: Context, ev: StepThreeEvent) -> StopEvent:
        # Wait until we receive 3 events
        result = ctx.collect_events(ev, [StepThreeEvent] * 3)
        if result is None:
            print("Not all events received yet.")
            return None

        # Do something with all 3 results together
        print(result)
        return StopEvent(result="Done")

```

```

w = ConcurrentFlow(timeout=10, verbose=False)
result = await w.run(message="Start the workflow.")
print(result)

```

```

Running query  Query 1
Running query  Query 2
Running query  Query 3
Not all events received yet.
Not all events received yet.
[StepThreeEvent(result='Query 3'), StepThreeEvent(result='Query 1'),
StepThreeEvent(result='Query 2')]
Done

```

In the above example, we emit three `StepTwoEvent` s which trigger `step_two` , each execution of which eventually emits a `StepThreeEvent` .

`step_three` gets triggered whenever a `StepThreeEvent` fires. What `collect_events` does is store the events in the context until it has collected the number and type of events specified in its second argument. In this case, we've told it to wait for 3 events.

If an event fires and `collect_events` hasn't yet seen the right number of events, it returns `None` , so we tell `step_three` to do nothing in that case. When `collect_events` receives the right number of events it returns them as an array, which you can see us printing in the final output. Note that in the array they are stored in the order they returned, not the order they were emitted.

To implement a map-reduce pattern, you would split your task up into as many steps as necessary, and use `Context` to store that number with `ctx.set("num_events", some_number)` . Then in `step_three` you would wait for the number stored in the context using `await ctx.get("num_events")` . So you don't need to know in advance exactly how many concurrent steps you're taking.



Collecting different event types

We don't just have to wait for multiple events of the same kind. In this example, we'll emit 3 totally different events and collect them at the end.

```
class StepAEvent(Event):
    query: str

class StepACompleteEvent(Event):
    result: str

class StepBEvent(Event):
    query: str

class StepBCompleteEvent(Event):
    result: str

class StepCEvent(Event):
    query: str

class StepCCompleteEvent(Event):
    result: str
```

```

class ConcurrentFlow(Workflow):
    @step
    async def start(
        self, ctx: Context, ev: StartEvent
    ) -> StepAEvent | StepBEvent | StepCEvent:
        ctx.send_event(StepAEvent(query="Query 1"))
        ctx.send_event(StepBEvent(query="Query 2"))
        ctx.send_event(StepCEvent(query="Query 3"))

    @step
    async def step_a(self, ctx: Context, ev: StepAEvent) -> StepACompleteEvent:
        print("Doing something A-ish")
        return StepACompleteEvent(result=ev.query)

    @step
    async def step_b(self, ctx: Context, ev: StepBEvent) -> StepBCompleteEvent:
        print("Doing something B-ish")
        return StepBCompleteEvent(result=ev.query)

    @step
    async def step_c(self, ctx: Context, ev: StepCEvent) -> StepCCompleteEvent:
        print("Doing something C-ish")
        return StepCCompleteEvent(result=ev.query)

    @step
    async def step_three(
        self,
        ctx: Context,
        ev: StepACompleteEvent | StepBCompleteEvent | StepCCompleteEvent,
    ) -> StopEvent:
        print("Received event ", ev.result)

        # Wait until we receive 3 events
        events = ctx.collect_events(
            ev,
            [StepCCompleteEvent, StepACompleteEvent, StepBCompleteEvent],
        )
        if (events is None):
            return None

        # Do something with all 3 results together
        print("All events received: ", events)
        return StopEvent(result="Done")

```

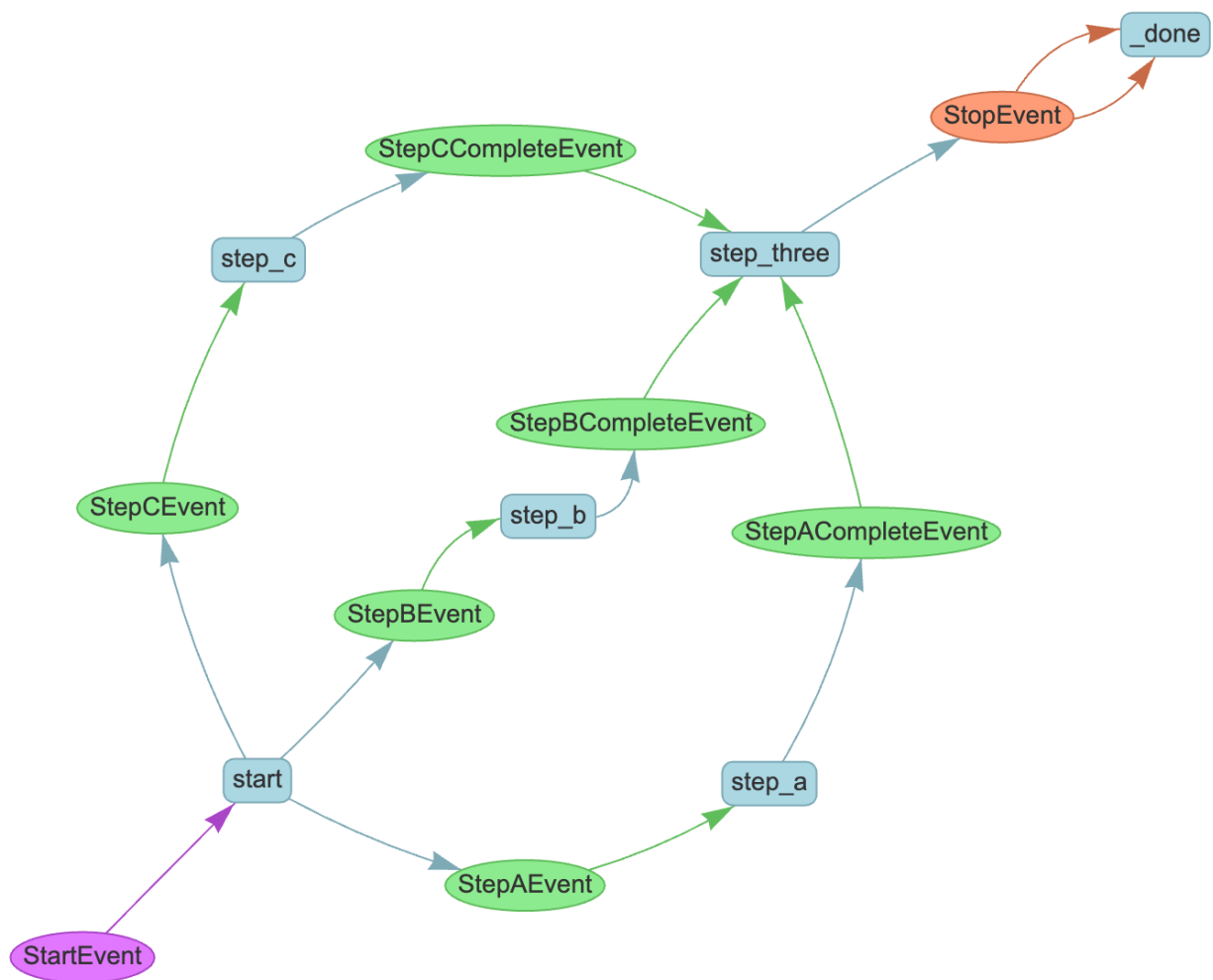
When we run, it will do all three things and wait for them in `step_three`.

```
w = ConcurrentFlow(timeout=10, verbose=False)
result = await w.run(message="Start the workflow.")
print(result)
```

```
Doing something A-ish
Received event  Query 1
Doing something B-ish
Received event  Query 2
Doing something C-ish
Received event  Query 3
All events received: [StepCCompleteEvent(result='Query 3'),
StepACompleteEvent(result='Query 1'), StepBCompleteEvent(result='Query 2')]
Done
```

Note that the order of the events is significant: `collect_events` was told to expect Step C, then A, then B, and that's the order they are in the return array.

These changes have a pretty pleasing visualization:



✨ Congratulations yet again!

You've now put together a Workflow from scratch, and learned the primitives necessary to build any workflow.